Graded Lab 3 Coverage Guided Fuzzing

libFuzzer Sanitizers Crash Analysis Program Repair

In this lab, you will apply the fuzzing workflow from the ungraded lab to an unfamiliar image-message parser. You will complete the harness, design the fuzzing inputs, find and minimize a memory-safety failure, minimize the corpus, and repair the program. The lab slides contain the command patterns you will need; this document describes the work and the required evidence without repeating those commands.

Learning objectives

- Explain how coverage feedback and corpus evolution guide mutation-based search.
- Write a libFuzzer harness that safely exercises a parser many times in one process.
- Design seed inputs and a dictionary for a structured input format.
- Use sanitizer output to reproduce, minimize, explain, and repair a memory error.
- Reduce a corpus while preserving the behavior that libFuzzer considers useful.

Image message format

The target accepts byte sequences that begin with IMG2 followed by semicolon-terminated key-value fields. Fields may appear in any order after the header. A representative input is shown below.

```
IMG2;FMT=GRAY;OP=INFO;AUTH=NO;W=2;H=3;DATA=abcdef;
```

Field	Meaning
IMG2	Required message header
FMT	Image format: GRAY, RGB, or META
OP	Requested operation: INFO, CHECK, or THUMB
AUTH	Authorization value: YES or NO
W and H	Decimal width and height from 1 through 64
DATA	Payload used by CHECK or THUMB

Files supplied

File	Purpose	Your responsibility
image.c and image.h	The parser under test and its public function	Understand the code and repair the memory error
image_cli.c	A normal program that reads one message	Use it for manual checks and reproduction
fuzz_target.c	The starter libFuzzer harness	Complete the missing harness behavior
Makefile	Build, regression-test, reset, and cleanup targets	Use the appropriate targets during the lab
tests folder	Normal accepted and rejected inputs	Keep these tests passing after the repair
seeds folder	Location for your initial corpus	Create your own small set of seed inputs
tokens.dict	Starter dictionary file	Add useful format tokens
lab_report.txt	Evidence and analysis template	Complete every section for submission

Required workflow

Complete the following stages in order. Save the requested observations in lab_report.txt as you work. Use separate corpus and artifact directories for the runs with and without a dictionary.

1 Understand the target

Read image.c, image.h, and image_cli.c. Trace how the parser locates fields, validates dimensions, chooses a format, and chooses an operation.

- Identify the conditions required for the INFO, CHECK, and THUMB paths.
- Record the important branches in the report before beginning the fuzzing campaign.

2 Complete the harness

Finish fuzz_target.c so every input supplied by libFuzzer is passed to the parser exactly once. The harness must not read files, print each input, terminate the process, or retain per-input state.

Explain why the harness is safe for repeated in-process execution.

3 Build and check the program

Build the normal, sanitizer-enabled, and fuzzing binaries using the patterns demonstrated in the slides. Resolve compiler errors and warnings caused by your work.

Run the provided normal-input tests. Confirm that the completed fuzz target starts correctly before collecting experimental results.

4 Design the seed corpus

Create a small initial corpus in the seeds folder. Use three to five concise, valid messages that exercise different formats or operations. Do not use random files as seeds.

- Each seed should have a specific purpose.

- Together, the seeds should give the fuzzer more than one useful starting path.
- Document which behavior each seed is intended to reach.

5 Run without a dictionary

Start with a fresh copy of your seed corpus and fuzz without a dictionary for the assigned duration. Keep the evolving corpus and crash artifacts from this run separate from later work.

Record the run duration, final coverage value, final feature value, final corpus size, and the number of crash artifacts in the report.

6 Create and use a dictionary

Study the input grammar and add useful literals to tokens.dict. Include structural markers and values that help mutations form meaningful messages. Do not place a complete crashing input in the dictionary.

Begin a new campaign from the same initial seeds, use the dictionary, and run for the same assigned duration. Record the same result fields as in the first campaign.

7 Reproduce and classify crashes

Inspect all crash artifacts produced by the campaigns. Group artifacts that report the same sanitizer error at the same relevant program location; these represent one unique crash cause.

Replay at least one artifact from every unique crash cause as a single input rather than as a new fuzzing campaign. Record the sanitizer error, the relevant function, and the evidence that the failure is reproducible.

8 Minimize one crash

Select one reproducible crash and use libFuzzer crash minimization to create a smaller input that preserves the same sanitizer failure.

- Save the minimized input as minimized-crash.img.
- Report the original and minimized byte counts.
- Explain which remaining fields or bytes are necessary to reach the failing path.

9 Minimize the corpus

Use libFuzzer corpus minimization to select a smaller subset of the dictionary-assisted corpus while preserving its useful coverage features.

- Store the selected files in work/min-corpus.
- Record the number of corpus files before and after minimization.
- Explain what corpus minimization preserves and why the result is not the same as minimizing a crash.

10 Repair and validate the program

Use the sanitizer report and the minimized crash to identify the memory-safety error in image.c. Repair the underlying size or bounds logic while preserving the intended parser behavior.

- Do not disable sanitizers, remove an operation, reject every input on the affected path, or hide the failure in the harness.

- Rebuild all binaries and rerun the provided normal-input tests.
- Replay the original and minimized crash inputs and confirm that neither produces a sanitizer error.
- Run a final sanitizer-enabled fuzzing campaign against the repaired program and report the result.

Submission requirements

Submit the complete graded_lab3 folder **(zipped)** with the following work included:

- Your completed fuzz_target.c.
- Your seed files in the seeds folder.
- Your completed tokens.dict.
- The original crash file created by libFuzzer and the smaller `minimized-crash.img` produced from it. Both files must reproduce the same sanitizer error.
- The minimized corpus in work/min-corpus.
- Your repaired image.c.
- A fully completed lab_report.txt with the requested evidence and explanations.

Repair standard

A repair is complete only when the normal behavior remains available, the regression tests pass, the saved crash inputs no longer trigger a sanitizer finding, and continued sanitizer-enabled fuzzing does not rediscover the same defect. **A workaround that suppresses the path, weakens the harness, or removes instrumentation does not receive repair credit.**